

StitchFlow AI: Technical Architecture & Documentation

Engineering Team

July 2026

1 Executive Summary

StitchFlow AI is an autonomous, multi-agent supply chain intelligence system designed for the fast-fashion sector. It replaces traditional, slow-moving procurement analysis with a swarm of specialized AI agents that dynamically cross-reference internal ERP data against live digital trends and local competitor intelligence.

2 System Architecture

The system is bifurcated into a Python-based backend agent swarm and a React-based frontend dashboard.

- **Agent Framework:** CrewAI (Python) orchestrates the agent swarm, managing sequential task delegation and data passing.
- **Data Sources:** Internal operations are mocked via JSON (`mock_erp.json`, `mock_local_market.json`), while real-world digital trend scraping feeds external context.
- **Output Artifacts:** The swarm produces two deterministic artifacts upon completion: a JSON feed for the live dashboard and a LaTeX-compiled PDF for executive sign-off.
- **Frontend:** A React application utilizing raw, vanilla CSS to enforce a strict "Cold Luxury / Industrial Brutalist" aesthetic.

3 Multi-Agent Swarm Design

The intelligence pipeline is distributed across three specialized agents, preventing monolithic hallucination and enforcing rigorous cross-validation.

3.1 1. ERP Inventory Analyst

Role: Internal logistics computation. **Capabilities:** Scans historical sales velocity to identify exponential demand curves (viral SKUs). Computes precise remaining warehouse capacity (m^3) and tracks the strictly enforced budget ceiling (e.g., 361,000 TND).

3.2 2. Trend Analyst

Role: External validation. **Capabilities:** Ingests digital scraper data to validate if an internal velocity spike is a true market trend (e.g., *Gorpcore*, *Quiet Luxury*) or a data anomaly.

3.3 3. Local Market Analyst

Role: Competitive intelligence. **Capabilities:** Audits local competitors (e.g., *TunisTech*, *SousseStyle*) to detect out-of-stock rates and price markups, identifying highly profitable gaps in the local supply chain.

4 Pipeline Execution

The pipeline is invoked via `main.py`.

1. **Data Ingestion:** The ERP Analyst reads the raw JSON datasets.
2. **Debate & Synthesis:** The Trend and Market analysts validate the ERP findings, dropping irrelevant SKUs.
3. **JSON Export:** A highly structured JSON payload (`data.json`) is written directly into the `dashboard/src/` directory, instantly updating the React application via Hot Module Replacement.
4. **PDF Compilation:** The Chief Reporter Agent drafts a professional procurement summary in LaTeX. The system intercepts this string, patches rendering bugs (e.g., `\approx`), and triggers `pdflatex` to compile a distributable PDF report into the `docs/` directory.

5 Deployment & Constraints

Safety Parameters

The system enforces strict mathematical cutoffs during the JSON export phase. If the agents attempt to procure items that exceed the remaining budget or available warehouse volume, the serialization engine immediately truncates the payload to prevent deficit spending.